

3.3 Coordinating the Machine's Activities

One of the most fundamental concepts of modern operating systems is the distinction between a program and the activity of executing a program. The former is a static set of directions, whereas the latter is a dynamic activity whose properties change as time progresses. (This distinction is analogous to a piece of sheet music, sitting inert in a book on the shelf, versus a musician performing that piece by taking actions that the sheet music describes.) The activity of executing a program under the control of the operating system is known as a **process**. Associated with a process is the current status of the activity, called the **process state**. This state includes the current position in the program being executed (the value of the program counter) as well as the values in the other CPU registers and the associated memory cells. Roughly speaking, the process state is a snapshot of the machine at a particular time. At different times during the execution of a program (at different times in a process), different snapshots (different process states) will be observed.

Unlike a musician, who normally tries to play only one musical piece at a time, typical time-sharing/multitasking computers are running many processes, all competing for the computer's resources. It is the task of the operating system to manage these processes so that each process has the resources (peripheral devices, space in main memory, access to files, and access to a CPU) that it needs, that independent processes do not interfere with one another, and that processes that need to exchange information are able to do so.

Process进程

- Each process represents a task
- Jobs vs Tasks
 - A collection of tasks that is used to perform a computation is known as a *job* (MSDN)
- Is a **process** the same as a **program**?

• What is a process

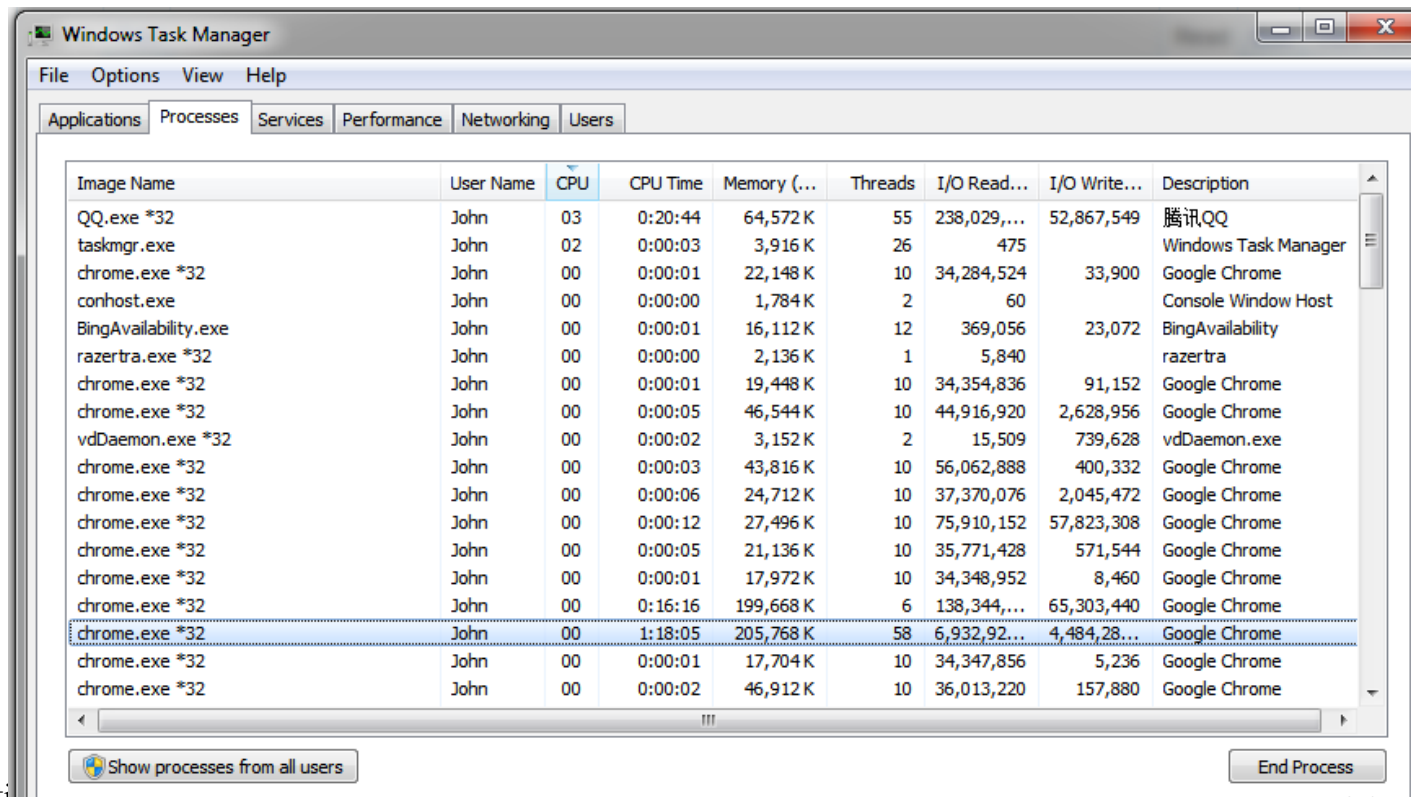


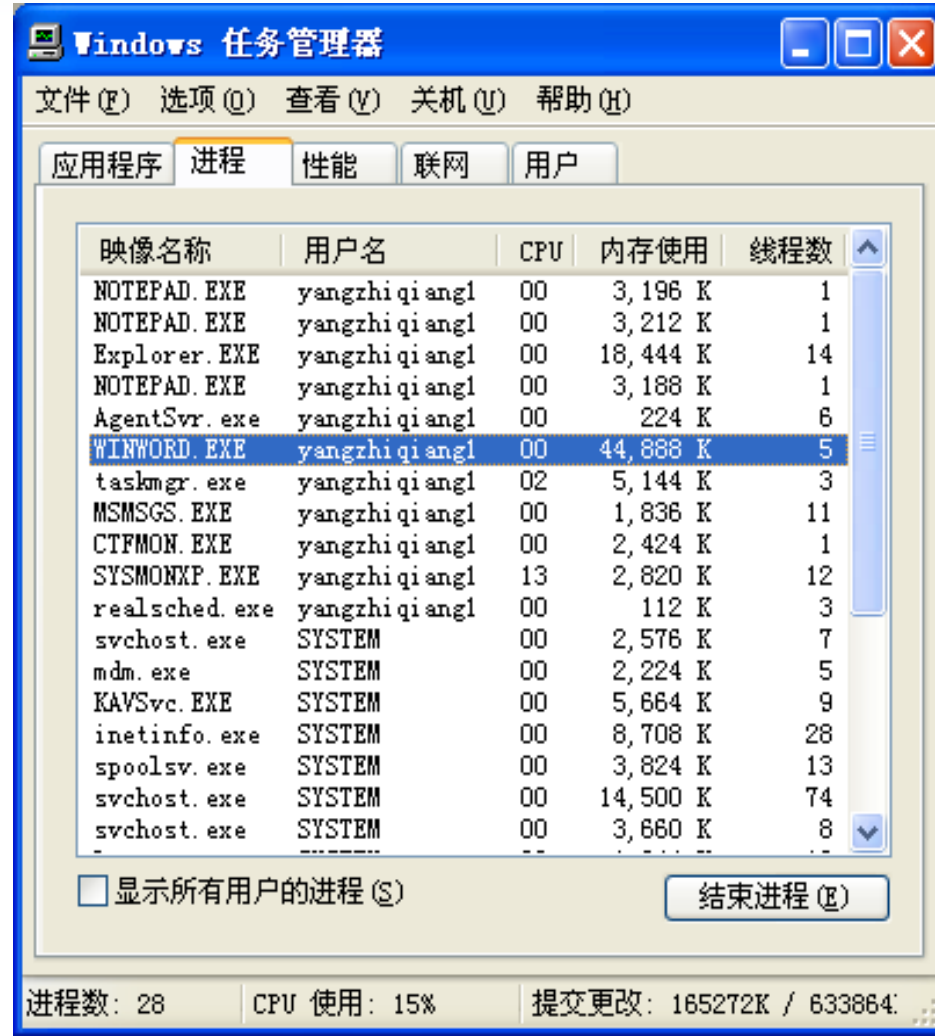
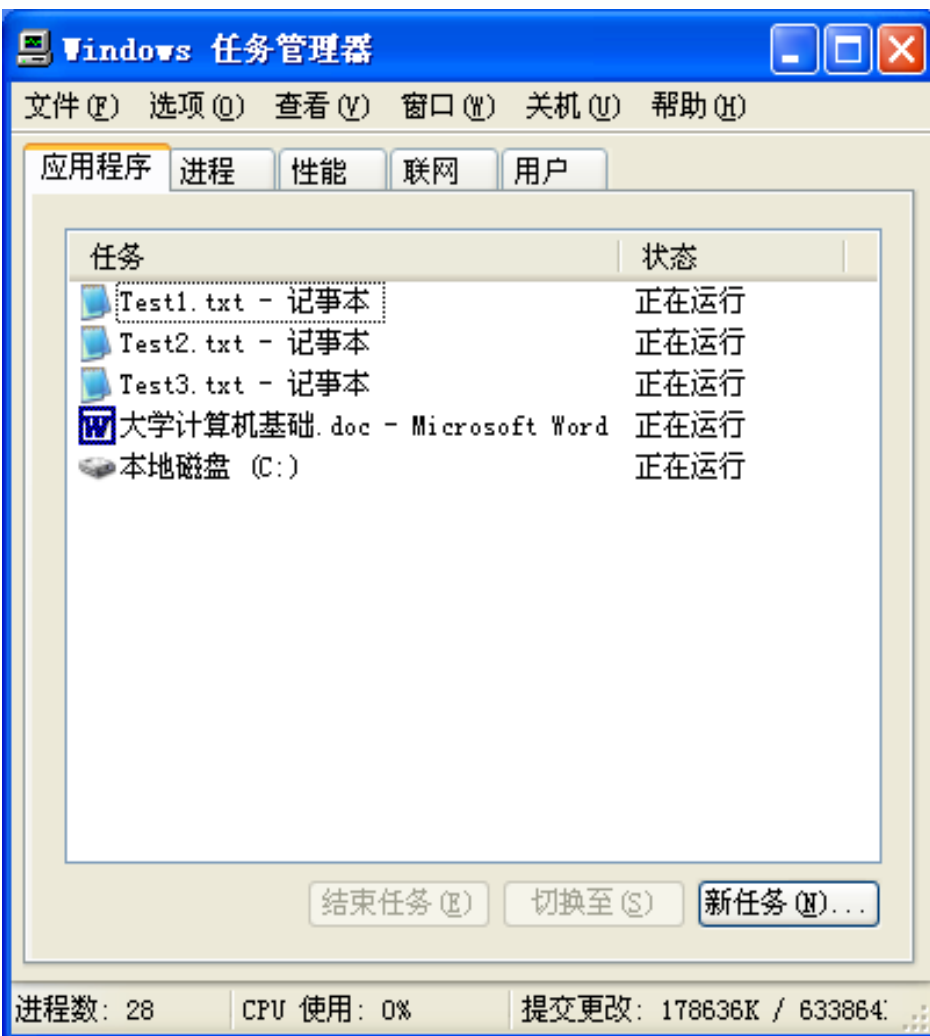
Image Name	User Name	CPU	CPU Time	Memory (...)	Threads	I/O Read...	I/O Write...	Description
QQ.exe *32	John	03	0:20:44	64,572 K	55	238,029,...	52,867,549	腾讯QQ
taskmgr.exe	John	02	0:00:03	3,916 K	26	475		Windows Task Manager
chrome.exe *32	John	00	0:00:01	22,148 K	10	34,284,524	33,900	Google Chrome
conhost.exe	John	00	0:00:00	1,784 K	2	60		Console Window Host
BingAvailability.exe	John	00	0:00:01	16,112 K	12	369,056	23,072	BingAvailability
razertra.exe *32	John	00	0:00:00	2,136 K	1	5,840		razertra
chrome.exe *32	John	00	0:00:01	19,448 K	10	34,354,836	91,152	Google Chrome
chrome.exe *32	John	00	0:00:05	46,544 K	10	44,916,920	2,628,956	Google Chrome
vdDaemon.exe *32	John	00	0:00:02	3,152 K	2	15,509	739,628	vdDaemon.exe
chrome.exe *32	John	00	0:00:03	43,816 K	10	56,062,888	400,332	Google Chrome
chrome.exe *32	John	00	0:00:06	24,712 K	10	37,370,076	2,045,472	Google Chrome
chrome.exe *32	John	00	0:00:12	27,496 K	10	75,910,152	57,823,308	Google Chrome
chrome.exe *32	John	00	0:00:05	21,136 K	10	35,771,428	571,544	Google Chrome
chrome.exe *32	John	00	0:00:01	17,972 K	10	34,348,952	8,460	Google Chrome
chrome.exe *32	John	00	0:16:16	199,668 K	6	138,344,...	65,303,440	Google Chrome
chrome.exe *32	John	00	1:18:05	205,768 K	58	6,932,92...	4,484,28...	Google Chrome
chrome.exe *32	John	00	0:00:01	17,704 K	10	34,347,856	5,236	Google Chrome
chrome.exe *32	John	00	0:00:02	46,912 K	10	36,013,220	157,880	Google Chrome

Processes: 178 CPU Usage: 6% Physical Memory: 83%

A Program vs. a Process

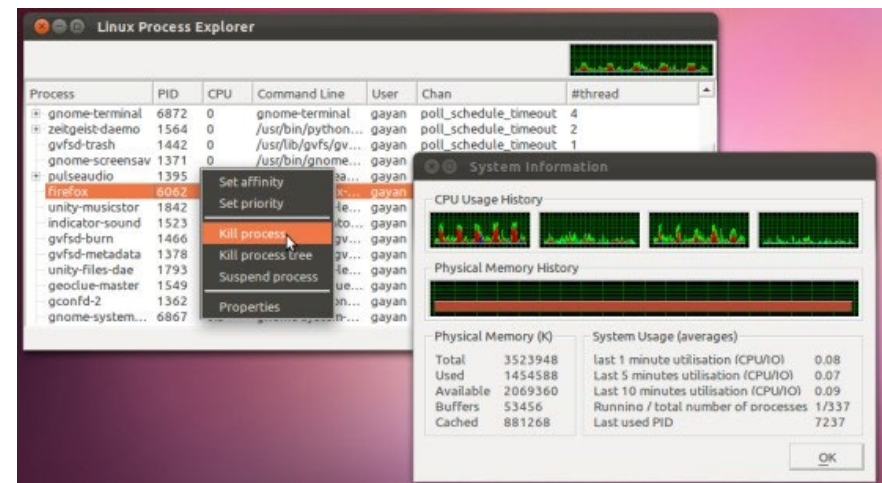
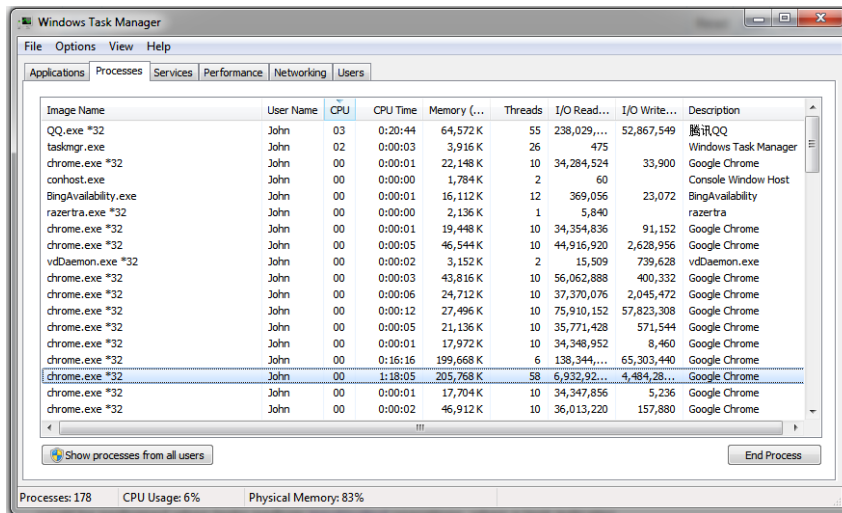
- **Program:** a set of instructions, e.g., notepad.c, notepad.exe
- **Process:** The activity of executing a program
- **Process State:** Current status of the activity
 - Program counter
 - General purpose registers
 - Related portion of main memory
- A program can be run multiple times, each instance/activity called a **process**
- Interprocess communication
 - The communication between processes (may from running one or more programs)

Processes



Process table

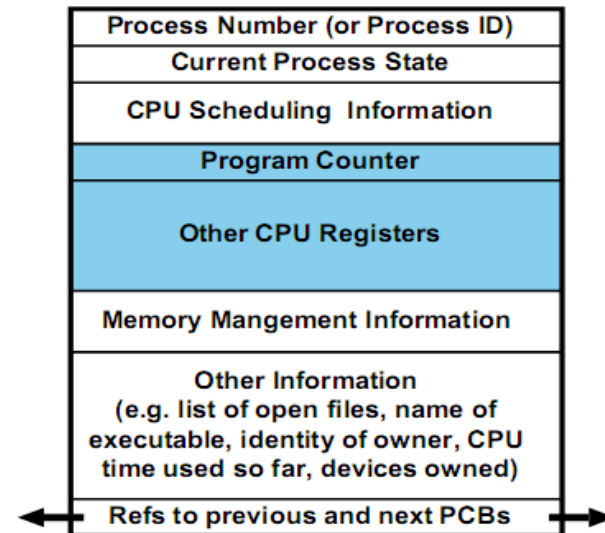
- A table of all the processes maintained by the operating system.
- Each entry is a process and its descriptions (PCB).



Process control block (PCB)

- “the manifestation (显示) of a process in an operating system”
 - Process identification
 - Processor state data (the status of a process, saved registers, program counter etc.)
 - Process control data (scheduling state, privileges etc.)

Process Control Block



The secret of concurrent execution

- What do you have to do when switching from one ongoing task to another?
 - Simply stop the current task and turn to another
 - Record the status of the current task and suspend it and turn to another
- Context switch

Context (Process State)

- Snapshot of current status of a process
 - A process identifier, or PID
 - Register values, Program Counter value
 - The memory space, I/O, files for the process
 - Can be saved and resumed as if the process is not interrupted
- Execution *state* of the process
 - Ready: ready for execution
 - Waiting: waiting for some I/O
 - Complete: finished process

Context switch

- The process of storing and restoring the state (context) of a process so that execution can be resumed from the same point at a later time.
- This enables multiple processes to share a single CPU and is an essential feature of a multitasking operating system.

Who is responsible for context switching?

- **Process management**
 - **Scheduler (调度)**: Adds new processes to the process table and removes completed processes from the process table
 - **Dispatcher (分派)**: Controls the allocation of time slices to the processes in the process table

When to switch?

- **Interrupt** a signal to the processor emitted by hardware or software indicating an event that needs immediate attention.
- The *interrupt handler* (part of dispatcher) starts after the interrupt to perform context switch
- Modern architectures are interrupt driven.
 - Software interrupt (I/O)
 - Hardware interrupt (press a key)

Process Administration

- **Scheduler:** Adds new processes to the process table and removes completed processes from the process table
- Determines which processes should be considered for execution based on some priorities or concerns
 - Using process table for administration
- Process state table
 - Ready or waiting
 - Priority
 - Memory area, etc.

Dispatcher

- **Dispatcher:** Controls the allocation of time slices to the processes in the process table
 - The end of a time slice is signaled by an interrupt.
- Gives time slices to a process that is ready
- Executes a **context switch** when the running process's time slice is over
 - *Time slice*: a time segment for each execution
 - *Interrupt*: the signal generated by a hardware timer to indicate the end of a time slice.
 - The *interrupt handler* (part of dispatcher) starts after the interrupt to perform context switch

Context Switch (process switch)

1. Get an interrupt from timer
2. Go to the interrupt handler
 - a. Save the context of process A
 - b. Find a process ready to run (Assume that is process B)
 - c. Load the context of process B
3. Start (continue) process B

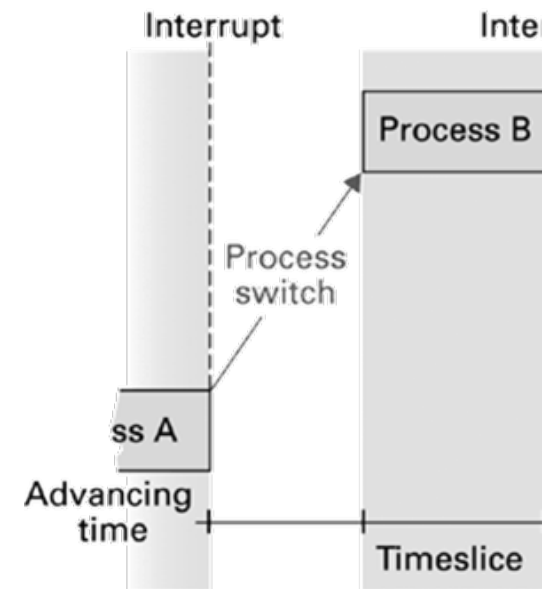
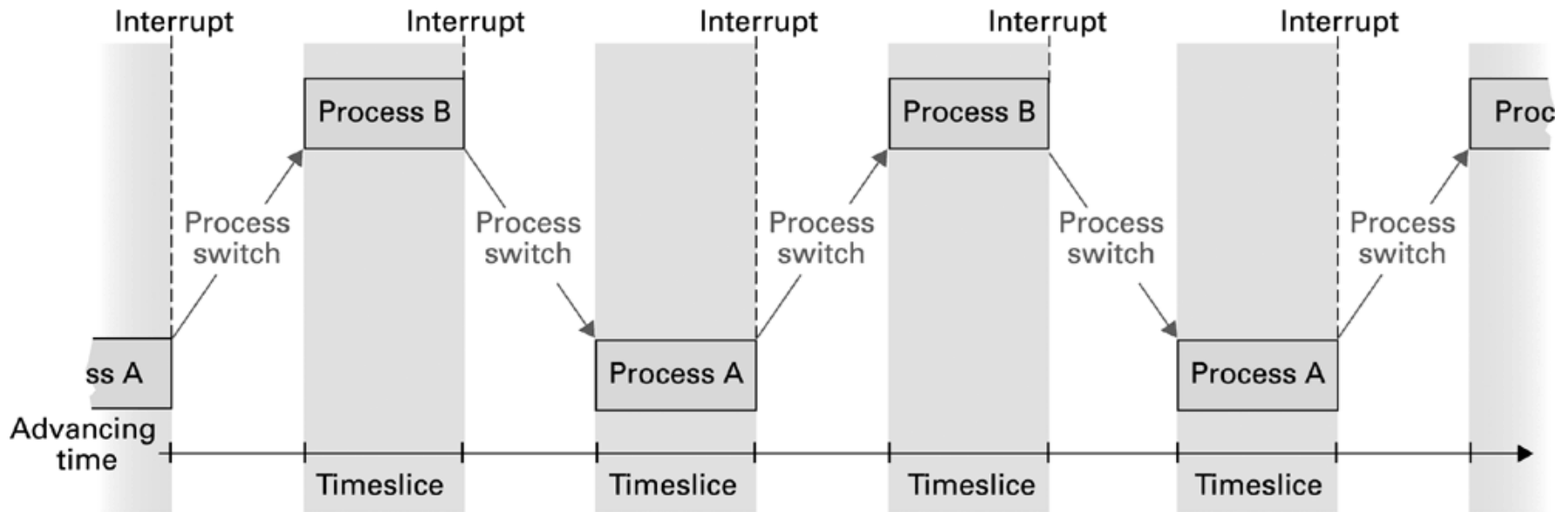
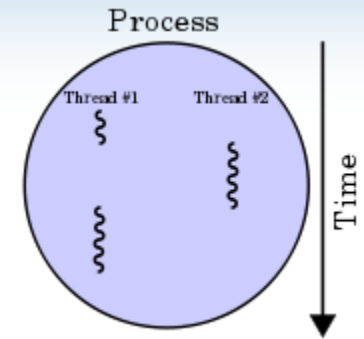


Figure 3.6 Time-sharing between process A and process B



Thread线程

- A task existing within a process that allows multiple independent instances to be executed concurrently
 - Multiple threads share resources such as memory, program code, ...
 - Each thread has its own program counter, registers, and stack (local memory)
- The context switch of threads is much faster than that of processes



文件(F) 选项(O) 查看(V) 关机(U) 帮助(H)

应用程序 进程 性能 联网 用户

映像名称	用户名	CPU	内存使用	线程数
NOTEPAD.EXE	yangzhiqiang1	00	3,196 K	1
NOTEPAD.EXE	yangzhiqiang1	00	3,212 K	1
Explorer.EXE	yangzhiqiang1	00	18,444 K	14
NOTEPAD.EXE	yangzhiqiang1	00	3,188 K	1
AgentSvr.exe	yangzhiqiang1	00	224 K	6
WINWORD.EXE	yangzhiqiang1	00	44,888 K	5
taskmgr.exe	yangzhiqiang1	02	5,144 K	3
MSMSG.S.EXE	yangzhiqiang1	00	1,836 K	11
CTFMON.EXE	yangzhiqiang1	00	2,424 K	1
SYSMONXP.EXE	yangzhiqiang1	13	2,820 K	12
realsched.exe	yangzhiqiang1	00	112 K	3
svchost.exe	SYSTEM	00	2,576 K	7
mdm.exe	SYSTEM	00	2,224 K	5
KAVSvc.EXE	SYSTEM	00	5,664 K	9
inetinfo.exe	SYSTEM	00	8,708 K	28
spoolsv.exe	SYSTEM	00	3,824 K	13
svchost.exe	SYSTEM	00	14,500 K	74
svchost.exe	SYSTEM	00	3,660 K	8

☐ 显示所有用户的进程(S)

结束进程(E)

查看(V) 帮助(H)

立即刷新(R)

更新速度(U)

选择列(S)...

选择列

请选择“任务管理器”进程页上将显示的列。

☒ 映像名称(I)

☐ PID (进程标识符)(E)

☒ CPU 使用(C)

☐ CPU 时间(E)

☒ 内存使用(M)

☐ 内存使用增量(U)

☐ 内存使用高峰值(K)

☐ 页面错误(E)

☐ USER 对象(U)

☐ I/O 读取

☐ I/O 读取字节

☒ 会话 ID(S)

☒ 用户名(U)

☐ 页面错误增量(A)

☐ 虚拟内存大小(V)

☐ 页面缓冲池(G)

☐ 非页面缓冲池(Q)

☐ 基本优先级(R)

☐ 句柄计数(H)

☒ 线程计数(T)

☐ GDI 对象(O)

☐ I/O 写入

☐ I/O 写入字节

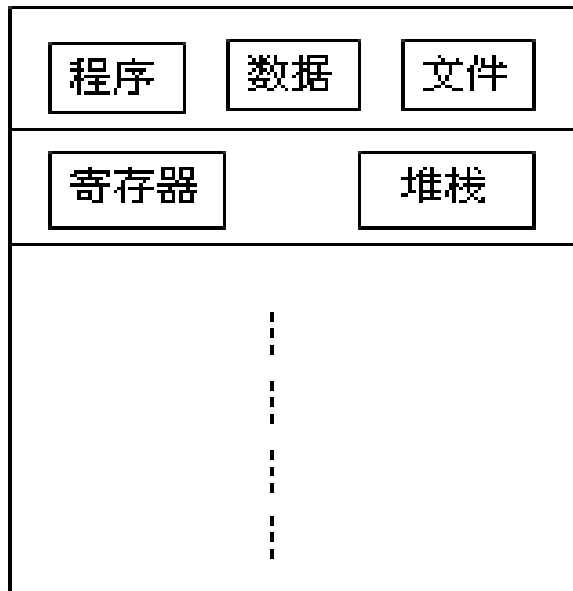
☐ I/O 其他

☐ I/O 其他字节

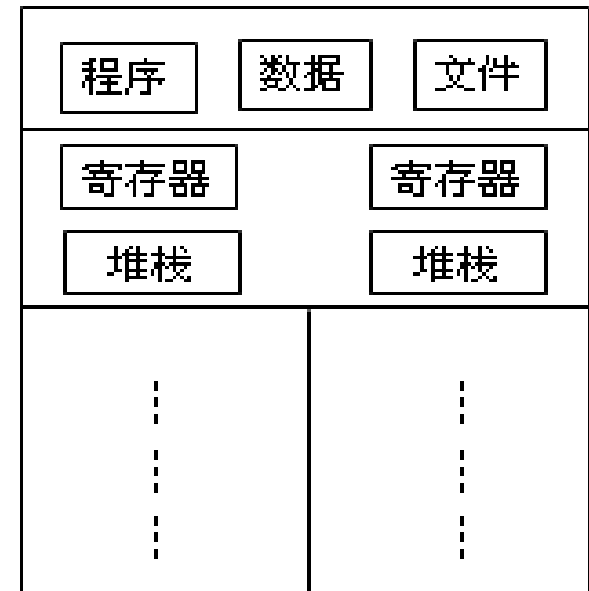
确定

取消

A Thread vs. a Process



单线程



多线程

Share more

High Concurrency

Fast Switch

***The operating system is itself a program** that is being executed on the same machine that it is controlling. In particular, such components as the command processor, file manager, or scheduler must essentially share time with the other processes in the system.

Operating System Architecture/Components of Operating System: Shell, File manager, Device drivers, Memory manager, Scheduler and dispatcher

In closing, we should note that the use of multiprogramming has been found to increase the overall efficiency of a machine. This is somewhat counterintuitive since the shuffling of processes required by multiprogramming introduces an overhead. However, without multiprogramming, each process runs to completion before the next process begins, meaning the time that a process is waiting for peripheral devices to complete tasks or for a user to make the next request is wasted. Multiprogramming allows this lost time to be given to another process. For example, if a process executes an I/O request, such as a request to retrieve data from a magnetic disk, the scheduler will update the process table to reflect that the process is waiting for an external event. In turn, the dispatcher will cease to award time slices to that process. Later (perhaps several hundred milliseconds), when the I/O request has been completed, the scheduler will update the process table to show that the process is ready, and thus that process will again compete for time slices. In short, progress on other tasks will be made while the I/O request is being performed, and thus the entire collection of tasks will be completed in less time than if executed in a sequential manner.

Questions & Exercises

1. What's the difference between Program and Process?
2. Advantages and disadvantage of multiprogramming or sequential manners?
3. Summarize the steps performed by the CPU when an interrupt occurs.
4. In a multiprogramming system, how can high-priority processes be allowed to run faster than others?
5. Suppose an OS allocates time slices in 10 millisecond units and the time required for a context switch is negligible. How many processes can obtain a time slice in one second?
6. If each time slice in a multiprogramming system is 50 milliseconds and each context switch requires at most a microsecond, how many processes can obtain the machine service in a single second? If each process uses its complete time slice, what fraction of the machine's time is spent actually performing processes?

Millisecond: 毫秒, $10^{-3}s$

Microsecond: 微秒, $10^{-6}s$

Nanosecond: 纳秒, $10^{-9}s$

3.4 Handling Competition Among Processes

Competition for Resources

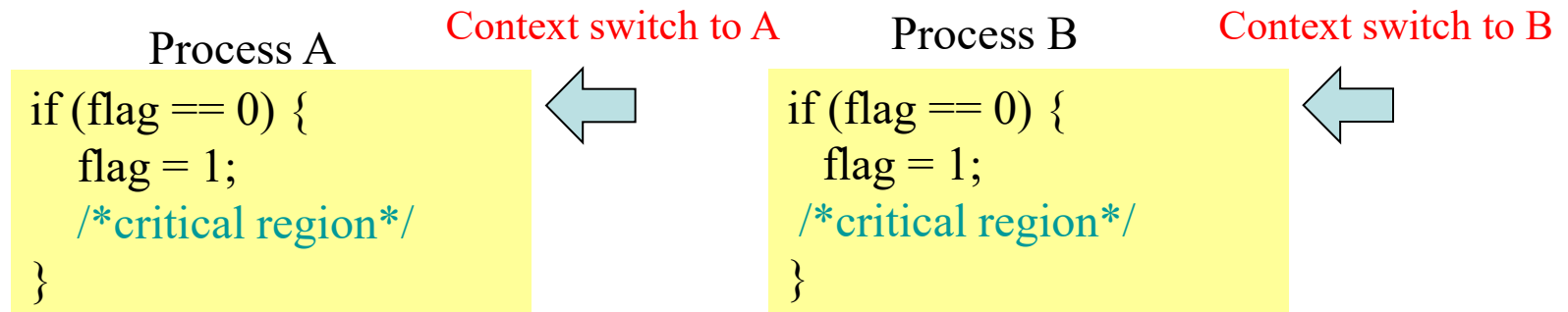
- What are *resources*?
 - CPU, memory, files, peripheral devices, ...
- In a multitasking system, resources are shared by processes
 - Some resources should not be employed by more than one process simultaneously
 - E.g., printer

Handling Competition for Resources

- **Semaphore:** A “control flag”
- **Critical Region:** A group of instructions that should be executed by only one process at a time
- **Mutual exclusion:** Requirement for proper implementation of a critical region

First Algorithm

- Use a flag (a global memory address)
 - flag=1: the critical region is occupied
 - flag=0: no process is in the critical region
- Problem:



- Both processes get into the critical region

Solutions

- Testing&setting the flag must be completed without interruption (atomic)
1. Use `disable_Interrupt()` to prevent context switch during the flag test and set process.
 2. A machine instruction called “test-and-set” which cannot be interrupted

```
Disable_Interrupt();  
if (flag == 0) {  
    flag = 1;  
    /*critical region*/  
}  
Enable_Interrupt();
```

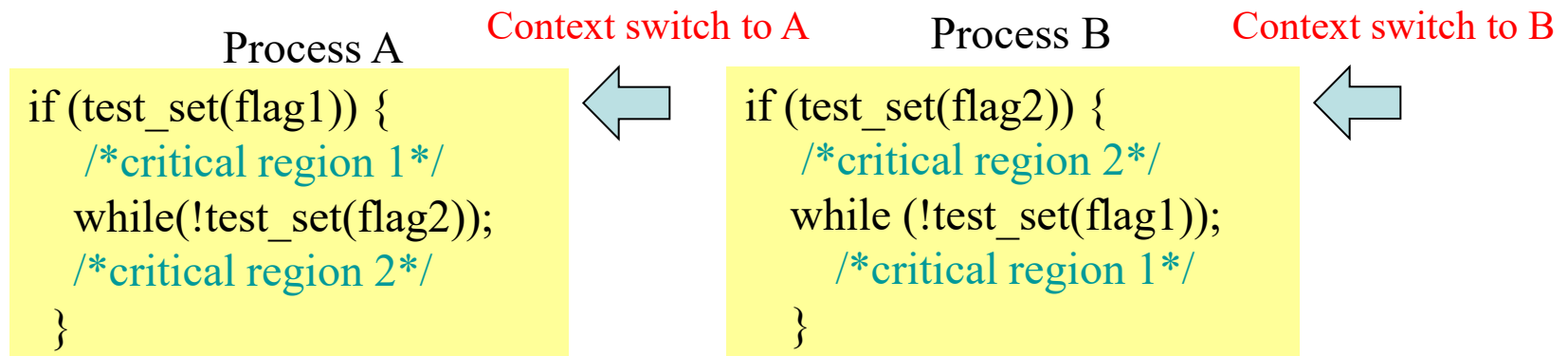
Questions & Exercises

For Multi-Core Operating Systems, what's the challenge for handling competition among processes?

Disabling interrupts on all cores whenever one needs to enter a critical region would be highly inefficient.

Another Problem: Deadlock

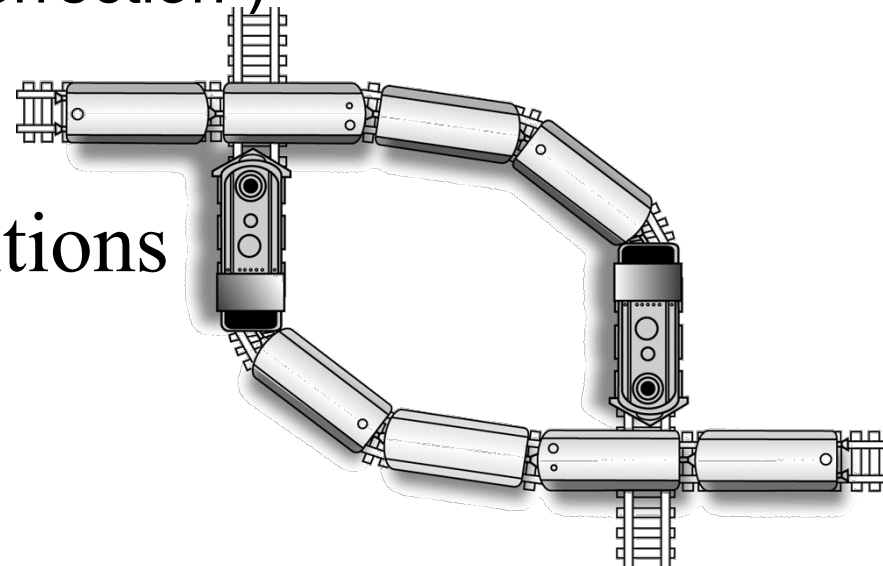
- Example:
 - A is in critical region 1, and waits to enter critical region 2
 - B is in critical region 2, and waits to enter critical region 1



Deadlock(Circular wait)

- Processes block each other from continuing
 - Conditions required for deadlock
 1. Competition for non-sharable resources
 2. Resources requested on a partial basis
 3. An allocated resource can not be forcibly retrieved (deadlock detection and correction)
- deadlock avoidance

Remove any one of the conditions can resolve the deadlock.



Techniques that attack the first two conditions are known as deadlock avoidance schemes. One, for example, attacks the second condition by requiring each process to request all its resources at one time. Another scheme attacks the first condition, not by removing the competition directly but by converting nonshareable resources into shareable ones. For example, suppose the resource in question is a printer and a variety of processes require its use. Each time a process requests the printer, the operating system could grant the request. However, instead of connecting the process to the printer's device driver, the operating system would connect it to a device driver that stores the information to be printed in mass storage rather than sending it to the printer. Thus, each process, thinking it has access to the printer, could execute in its normal way. Later, when the printer is available, the operating system could transfer the data from mass storage to the printer. In this manner, the operating system would make the nonshareable resource appear shareable by creating the illusion of more than one printer. This technique of holding data for output at a later but more convenient time is called **spooling**.

Multi-Core Operating Systems

Traditional time-sharing/multitasking systems give the illusion of executing many processes at once by switching rapidly between time slices faster than a human can perceive. Modern systems continue to multitask in this way, but in addition, the latest multi-core CPUs are genuinely capable of running two, four, or many more processes simultaneously. Unlike a group of single-core computers working together, a multi-core machine contains multiple independent processors (in this case called cores) that share the computer's peripherals, memory, and other resources. For a multi-core operating system, this means that the dispatcher and scheduler must consider which processes to execute on each core. With different processes running on different cores, handling competition among processes becomes more challenging because disabling interrupts on all cores whenever one needs to enter a critical region would be highly inefficient. Computer science has many active research areas related to building operating system mechanisms better suited to the new multi-core world.

Questions & Exercises

Solutions: Which condition is removed?

1. Kill one of the process
2. Processes need to request all the required resources at one time
3. Divide a file into pieces so that it can be altered by different processes
4. Spooling
 - For example, stores the data to be printed and waits the printer available

Questions & Exercises

Suppose process A and process B are sharing time on the same machine, and each needs the same nonshareable resource for short periods of time. (For example, each process may be printing a series of independent, short reports.) Each process may then repeatedly acquire the resource, release it, and later request it again. What is a drawback to controlling access to the resource in the following manner:

Begin by assigning a flag the value 0. If process A requests the resource and the flag is 0, grant the request. Otherwise, make process A wait. If process B requests the resource and the flag is 1, grant the request. Otherwise, make process B wait. Each time process A finishes with the resource, change the flag to 1. Each time process B finishes with the resource, change the flag to 0.

另外一个进程若无需使用，
则无法改变FLAG的值。

Questions & Exercises

Suppose the following solutions have been proposed for removing the deadlock that occurs on a single-lane bridge when two cars meet. Identify which condition for deadlock given in the text is removed by each solution.

- a.** Do not let a car onto the bridge until the bridge is empty.
- b.** If cars meet, make one of them back up.
- c.** Add a second lane to the bridge.

What's the drawback of solution a.?

Questions & Exercises

Suppose we represent each process in a multiprogramming system with a dot and draw an arrow from one dot to another if the process represented by the first dot is waiting for a (nonshareable) resource being used by the second. Mathematicians call the resulting picture a **directed graph**. What property of the directed graph is equivalent to deadlock in the system?

We have introduced spooling as a technique for granting several processes access to a common resource—a theme that has many variations. For example, a file manager could grant several processes access to the same file if the processes are merely reading data from the file, but conflicts can occur if more than one process tries to alter a file at the same time. Thus, a file manager may allocate file access according to the needs of the processes, allowing several processes to have read access but allowing only one to have write access. Other systems may divide the file into pieces so that different processes can alter different parts of the file concurrently. Each of these techniques, however, has subtleties that must be resolved to obtain a reliable system. How, for example, should those processes with only read access to a file be notified when a process with write access alters the file?

3.5 Security

- Attacks from outside
 - Problems
 - Insecure passwords
 - Sniffing software
 - Record and report
 - Simulate login procedure to trick authorized user
 - Counter measures
 - Auditing software-user behavior

Security (continued)

- Attacks from within
 - Problem: Unruly processes
 - Counter measures:
 - special-purpose with upper and lower limits of a process's allotted memory area
 - Control process activities via privileged modes and nonprivileged instructions to limit the changing the special-purpose registers
 - Nonprivilege mode: only “safe” instructions
 - For example, to access some part of memory.
 - Privilege mode: all kinds of instructions
 - Those instructions that can be only executed in the privilege mode are called **privilege instructions**.

Multi-Factor Authentication

One solution to the pervasive problem of humans using weak password to secure their computer accounts is to require more than one piece of evidence to gain access. Passwords are an example of something that a user *knows*, along with the answers to commonly used security questions, such as “What is your maternal grandmother’s maiden name?” A second authentication factor might depend on something that the user *has*, such as an ATM card, a specialized security token, or a smartphone with a particular phone number. A growing number of online accounts require *two-factor authentication*, in which the user must provide a password, and also a special access code text messaged to the user’s smartphone during the login process. A third set of authentication factors might depend on something related to who the user *is*. Biometric sensors embedded in mobile devices are now able to readily recognize a user’s unique fingerprint, voice, or retinal pattern. Some researchers are working on additional biometric mechanisms that might, for example, use smartphone accelerometer data to recognize a user’s unique walking gait pattern.

None of these mechanisms has proven to be entirely foolproof, however, as clever attackers have employed schemes to snoop user passwords, fool embedded sensors, or even trick human administrators into changing the smartphone number associate with another user’s account.

Foolproof: 万无一失的； 完全可靠的； 使用简便的

重点掌握内容

- 1. 名词解释：批处理（Batchprocessing）和交互式处理（Interactiveprocessing），分时（Time-sharing）和多道程序（Multiprogramming），壳和内核（Shell and Kernel in OS），进程和程序（Process and Program），线程（Thread），中断（Interrupt），死锁（DeadLock）
- 2. 操作系统的功能（Functions of OS）
- 3. 内核的功能和组成（Kernel）
- 4. 上下文切换（Contextswitching）

一般掌握内容

- 1. 启动的过程 (The Booting process)
- 2. 虚拟内存和页面调度 (Virtual memory and paging)
- 3. 调度器和分派器 (Scheduler and Dispatcher)
- 4. 资源竞争处理 (Handling Competition for Resources)

一般了解内容

- 1. 软件的分类 (Software classification)
- 2. 抢占式多任务与合作式多任务
(Preemptive vs Cooperative Multi-tasking)
- 3. 共享处理的发展 (Evolution of Shared Computing)
- 4. 信息安全 (Security)
- 5. 权限控制 (Privilegecontrol)